

高级语言程序设计

1

复习

理论课程



福州大学
FUZHOU UNIVERSITY

C语言简介



最简单的程序

- 最简单的规范程序（计算机视角）

C源代码由一个或多个函数构成。
可执行程序从主函数开始执行

```
int main()
```

第1行说明这是主函数，类型是整型，圆括号说明这是一个函数。

```
{
```

第2行说明这个函数从这里开始

```
    return 0;
```

第3行是一个返回语句，程序运行到主函数的返回语句停止。

```
}
```

第4行说明主函数到这里结束

这个程序什么也不做，只是向调用它的程序返回0。

最简单的程序

1-13 分数 2 查看题目详情

C语言程序是从源文件的第一条语句开始执行的。

☐ T

☐ F

F。

C 程序的执行起点是 `main` 函数 的第一条语句，而不是源文件的第一条语句。因此，描述是错误的。

简单的缩进示例

- 打印4次“ Hi! ”的程序

```
#include <stdio.h>
```

```
int main()  
{
```

函数体的花括号与函数头同缩进

```
    int i;
```

语句体花括号内的语句降一级，增加缩进四个空格

```
    for (i = 0; i < 4; i++)  
        printf("Hi!\n");
```

语句默认与上一条语句同级

循环语句内的语句降一级

```
    return 0;  
}
```

循环语句结束后恢复原有级别

说明: #include 指令与头文件

- #include是C预编译器指令的一个例子
 - 编译前对源代码进行一些预备工作, 称为预编译
 - 作用: 将文件完整内容复制到#include出现的位置
 - 预编译器指令以#开头, 不是语句, 不加分号
- 包含文件提供了一个共享信息的简便方法
 - 函数应先声明后使用, 类似列方程前需要设未知数
 - 函数的声明往往由一组语句构成, 很难默写; 而记住文件名, 则容易得多。

说明: #include 指令与头文件

```
#include <stdio.h> /* Standard I/O header */
```

```
int main(char *argv, int argc)
{
```

```
    double v0 = 0; //initial velocity
    double a = 0; //acceleration
    double t = 0; //time
    double s = 0; //shift
    printf("Enter the Initial Velocity (m/s): ");
    scanf("%lf", &v0);
    printf("Enter the Acceleration (m/(s*s)): ");
    scanf("%lf", &a);
```

头文件包含
在这里展开

```
00000140: 00 01 11 00 00 00 00 44 3A 5C 44 65 73 6B 74 6F
00000150: 70 5C 73 68 69 66 74 2E 6F 62 6A 00 3A 00 3C 11
00000160: 00 22 00 00 07 00 12 00 00 00 0D 52 01 00 12 00
00000170: 00 00 0D 52 01 00 4D 69 63 72 6F 73 6F 66 74 20
00000180: 28 52 29 20 4F 70 74 69 6D 69 7A 69 6E 67 20 43
00000190: 6F 6D 70 69 6C 65 72 00 00 00 00 45 6E 74 65 72
000001A0: 20 74 68 65 20 49 6E 69 74 69 61 6C 20 56 65 6C
000001B0: 6F 63 69 74 79 20 28 6D 2F 73 29 3A 20 00 00 25
000001C0: 6C 66 00 45 6E 74 65 72 20 74 68 65 20 41 63 63
000001D0: 65 6C 65 72 61 74 69 6F 6E 20 28 6D 2F 28 73 2A
000001E0: 73 29 29 3A 20 00 00 25 6C 66 00 45 6E 74 65 72
```

```
.....D:\Deskt
p\shift.obj...<
..R... R..
.. R.Microsoft
(R) Optimizing C
ompiler...Enter
the Initial Vel
ocity (m/s): ...%
lf.Enter the Acc
eleration (m/(s*
s)): ...%lf.Enter
```

```
00000000: 4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00
00000010: B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000030: 00 00 00 00 00 00 00 00 00 00 00 00 00 D8 00 00
00000040: 0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68
00000050: 69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F
00000060: 74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20
00000070: 6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00
00000080: 4A EC B7 36 0E 8D D9 65 0E 8D D9 65 0E 8D D9 65
00000090: D3 72 12 65 0D 8D D9 65 0E 8D D8 65 43 8D D9 65
```

```
MZ?.....??..
?.....@.....
.....
.....?...
?..?..?..L?!Th
is program canno
t be run in DOS
mode....$.....
J??6???e???e
?re.???eC???e
```

Source Code (.c)

源代码

Compiler
编译器

Object Code (.obj)

目标代码

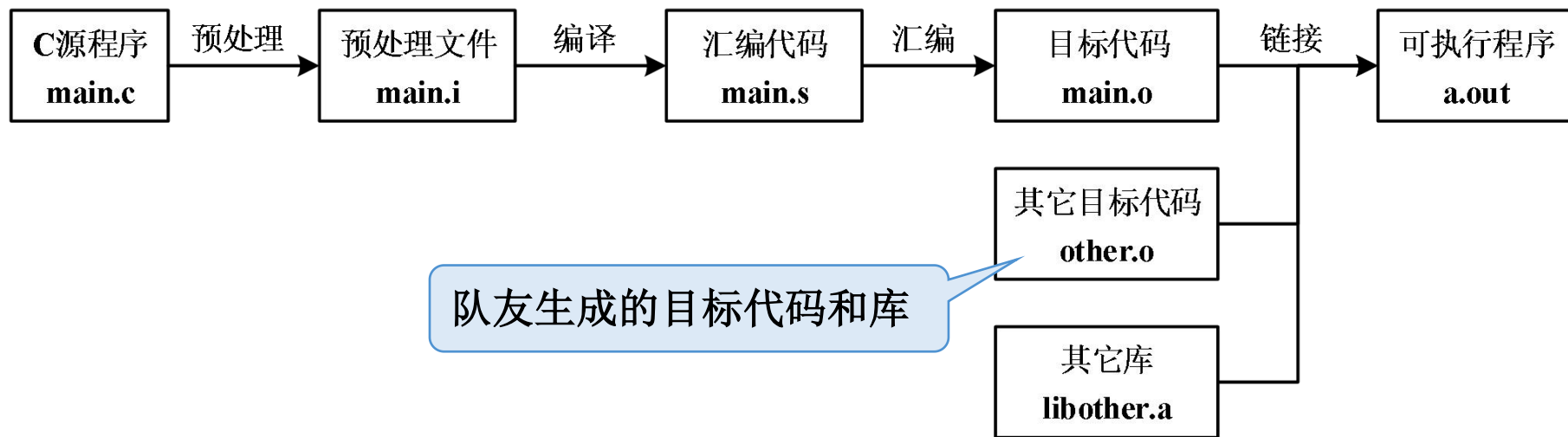
Linker
链接器

Executable Code (.exe)

可执行代码

编译程序

- 编译（Compile）：利用编译程序从源语言编写的源程序产生目标程序的过程。
- 编译器（Compiler）：执行编译功能的程序
- 生成一个C程序可以分为四阶段



编译程序

1-8 分数 2 查看题目详情

C语言中的预处理命令都以“#”开头。

☐ T

☐ F

1-15 分数 2 查看题目详情

C语言源代码可以不经编译而被计算机直接运行。（ ）。

☐ T

☐ F

F。

C 是一种编译型语言，其源代码需要被编译器转化为计算机可以理解的机器语言（即二进制代码）。编译成功后会生成可执行文件（如在 Windows 上的 .exe 文件），然后才可以运行。

T。

在 C 语言中，预处理命令是以“#”开头的语句，为编译器提供预处理指令。

型的预处理指令：

常见的预处理命令包括：

#include：用于引用头文件；

#define：用于宏定义；

#ifdef / #ifndef：条件编译语句；

#endif：结束条件编译块；

其他命令，如 #undef、#pragma 等。

常用的编译环境

- **Visual Studio C 编译器：**对C标准遵循不严格
- **GCC编译器：**遵循C标准
 - **Windows系统：**MinGW，Cygwin
 - **Linux系统：**GCC
- **不同的编译器对C标准未规定的部分处理不一样**

命名选择：合规规则

- 合规（valid）的命名包括
 - 小写字母、大写字母、数字、下划线（_）
 - 不可与保留字一致

合规命名	非法命名
wiggles	<code>\$Z]**</code>
Cat2	<code>2cat</code>
Hot_Tub	<code>Hot-Tub</code>
taxRate	<code>tax rate</code>
_kcab	<code>don't</code>

C语言的关键字

```
auto extern short while break
float signed case for sizeof
char goto static const if struct
continue inline switch default
int typedef do long union double
register unsigned else restrict
void enum return volatile
_Alignas _Alignof _Bool _Complex
_Generic _Imaginary _Noreturn
_Static_assert _Thread_local
```

命名选择：合规规则

- C命名大小写敏感（case sensitive），但不可混用
 - 大小写字母含义不同（stars与Stars、STARS不同）
- 命名长度
 - 传统编译器只识别前63个字符，现在编译器会重命名处理。

但不应同时出现在同一段代码里。读者会混淆：是不是同一个东西？
- 选取合适的变量或函数、文件名称具有重要意义
 - 它使得阅读代码的程序员能迅速掌握代码的意图
 - 错误或随意的命名迷惑或误导读者，队友不愿意和你合作
 - 思考合理命名的过程，也是理清编程思路的过程

命名选择：合规规则

1-2 分数 2 查看题目详情

下划线是标识符中的合法字符。

☐ T

☐ F

1-20 分数 2 查看题目详情

C语言标识符中，大小写字母是有区别的。

☐ T

☐ F

T。

在 C 语言中，标识符可以包含字母、数字和下划线，且不能以数字开头。

T。

C 语言是区分大小写的，标识符例如 Variable 和 variable 是不同的。

注释 (Comments)

- 封装于 `/* */` 符号的部分是注释 (comments)
 - 此种情况下可以换行，但起始标记不可没有结束标记 (`*/`)

```
/* This is a C comment. */  
/* This comment, being somewhat wordy, is spread over  
   two lines. */  
/*  
   You can do this, too.  
*/
```

```
/* But this is invalid because there is no end marker.
```

- 使用 `//` 可以增加一行注释

```
// Here is a comment confined to one line.  
int rigue; // Such comments can go here, too.
```

注释：要求

- 注释应在言之有物前提下尽量丰富
 - 尊重读者：太简单的代码不应注释，不应误导读者
 - 注释做什么和为什么，而不是怎么做
 - 不应用注释代替变量命名规范
- 需要和它人分享的函数应详细说明
 - 用途、输入参数列表及其作用、返回值类型及其意义等
 - 作者的姓名和联系方式、日期、版本号、版权声明

注释 (Comments)

1-4 分数 2 查看题目详情

过多的注释会影响程序的运行效率。

☐ T

☐ F

F。

注释在编译时不会包含到最终的机器代码中，因此不会影响程序的运行速度和效率。

数据类型



数学和计算机中的变量

- 数学和物理学中的变量（variable）

- 表达式或公式中，没有固定的值而可以变动的数或量

随自变量变化的量 $y = f(x)$ 自变量，自由变化的量

- 计算机中的变量

- 抽象的存储地址，它含有

- 值：某种已知或未知的信息量
- 名称：配对了关联的符号名称

```
int x = 20;
```

- 通常使用变量名称引用存储值

变量的属性

- 计算机中的变量是抽象的存储地址 `int weight = 20;`

类别	属性	值（示例）
基本	名称	weight
	值	20
	物理意义	重量
	数据类型	整型
内存相关	地址	0x3000~0x3003
	长度	4字节
	电平表示	十六进制： 0x14, 0x00, 0x00, 0x00 二 进 制： 0010 1000 000.....00

变量的写入值

• 变量写入值的方式

方式	说明	格式	示例
赋值	利用赋值运算符改变左值的值	<变量名> = <表达式>	<code>var = 65;</code>
写内存	调用scanf和memset等函数写入内存改变值	<函数名> (<参数列表>)	<code>scanf("%f", &var);</code>

取地址操作符&

• 赋值

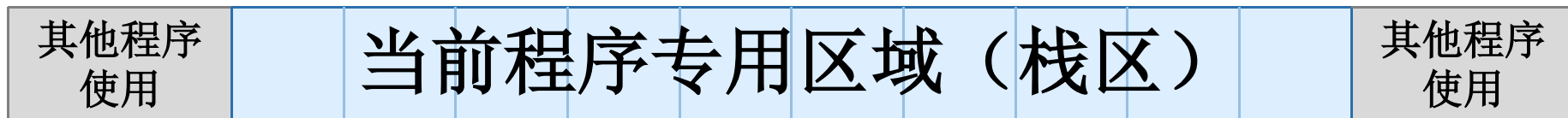
– 按优先级计算，用右值改变左值

• 写内存（以scanf函数为例）

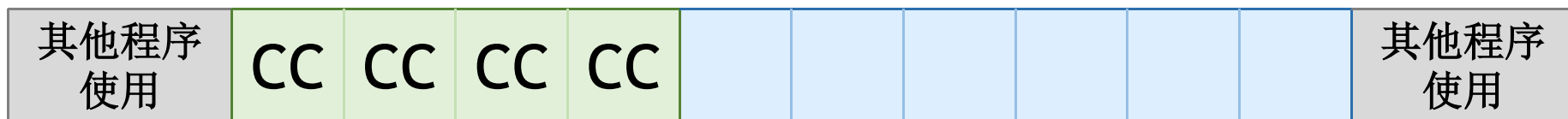
- 按格式字符串解析用户输入的内容，赋值到后续地址列表
- 一般变量应取地址，字符串名不取地址

变量相关的内存变化

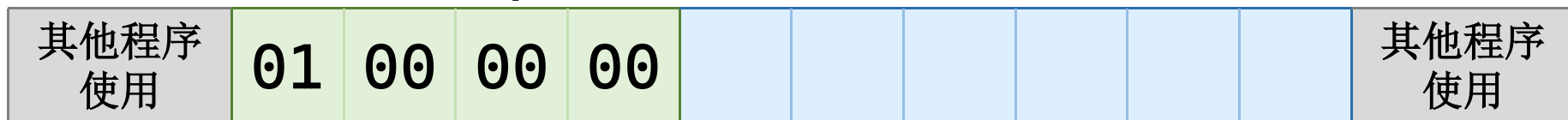
- 原始的内存空间



- 变量声明 (`int a;`) 时，开辟4字节空间并关联名称
a所在区域



- 变量赋值 (`a=1;`) 时，改变内存内容



- 变量求值 (`a`) 时，查询对应内存区域的内容（1）

变量

1-1 分数 2 查看题目详情

变量被定义后，它不仅有一个确定的地址值，而且还会有一个确定的本身值。

☐ T

☐ F

F。

变量被定义后，系统会为它分配一个内存地址（即地址值）。但变量的值只有在初始化后才是确定的，否则其值是未定义的或随机的，无法保证是一个确定的值。

常量

- 常量（Constants）：程序运行过程中保持值不变的量
 - 例如：程序运行中 $\pi = 3.14$ 是不能随便改动的
- 常量的声明

```
const double PI = 3.14;
```

 - 常量名一般不使用小写字母
- 常量具备变量的一切属性和方法
 - 仅允许声明时赋值，此后常量名不得作为赋值表达式左值

修改常量产生编译错误

- 尝试修改const常量将产生编译错误

```
const float mult;  
mult = 23 * weight * 14.5833;
```

黄色底色的方框中是
错误的示例代码

- 尝试修改常量

- 编译时出现错误：为只读变量（常量）赋值

```
const.c: In function 'main':  
const.c:9:5: error: assignment of read-only variable 'mult'  
    mult = 23;  
    ^
```

绿色底色的方框中是
编译器的输出信息

- 对#define常量或数值常量赋值也将产生编译错误

- 宏是简单替换，不因宏定义而产生额外的空间

常量和宏

- 常量（Constants）：运行过程中保持值不变的量
- 用宏定义常量：为特殊意义常数取一个好记的名称
- 常量占用存储空间，宏和常数不开辟专门的存储空间

分类	示例	格式
常数	14.5833	整数、实数、字符、字符串，或者八进制、十六进制。
宏定义	<code>#define PP0 14.5833</code>	预编译指令 <code>#define</code> ，宏名，宏的展开
常量	<code>const float X = 1700.0;</code>	关键字 <code>const</code> ，数据类型，变量名，赋值符号，值。

常量和宏

1-9 分数 2 查看题目详情

`#define MAX 100` 和 `int MAX=100;` 的效果相同，都是定义一个变量 `MAX` 并使其值为100。

☐ T

☐ F

F。
`#define MAX 100` 是一个预处理指令，用于定义一个宏，它在编译前会被替换为字面值 100，并不真正占有内存，是无类型的。
`int MAX = 100;` 是定义了一个变量，它占据内存，并且是 `int` 类型，可改变值。

常量和宏

38、若程序中有宏定义行：

```
#define N 100
```

则以下叙述中正确的是

- ☐ A. 宏定义行中定义了标识符N的值为整数100
- ☐ B. 在编译程序对C源程序进行预处理时用100替换标识符N
- ☐ C. 上述宏定义行实现将100赋给标识符N
- ☐ D. 在运行时用100替换标识符N

B。

宏定义只是一个简单的文本替换，它不会给 N 赋值

宏不是变量，不能用等号赋值。#define 不涉及实际的值赋给变量的操作，纯粹是进行文本替换。

宏替换发生在编译的预处理阶段，而不是运行时。

预定义为#define的常量

- 格式 `#define NAME value // comments`
- 行为：预编译器替换所有关键字，再交给编译器编译
 - 文中除注释和字符串外所有完整的NAME都替换为value

使用宏定义的语句	宏展开的语句
<code>circum = 2.0 * PI * radius;</code>	<code>circum = 2.0 * 3.14159 * radius;</code>
<code>"PI is 3.14"</code>	<code>"PI is 3.14"</code>
<code>area = PIs * radius * radius;</code>	<code>area = PIs * radius * radius;</code>
宏定义	s = PI * r * r; 宏展开的语句
<code>#define PI 3.1415</code>	<code>s = 3.1415 * r * r;</code>
<code>#define PI 3 + 0.1415</code>	<code>s = 3 + 0.1415 * r * r;</code>
<code>#define PI = 3.1415</code>	<code>s = = 3.1415 * r * r;</code>

整数常数的表示

- 格式（大小写无关） [正负号]<十/八/十六进制常数>[整数后缀]

正负号	可省略	示例（106）
正号（默认）	是	+106 或 106
负号	否	-106

进制序列	前缀	数字字符集	示例（106）
十进制	无	0123456789	106 ($=1*10^2+0*10^1+6*10^0$)
八进制	0	01234567	0152 ($=1*8^2+5*8^1+2*8^0$)
十六进制	0x	0123456789ABCDEF	0x6a ($=6*16^1+10*16^0$)

后缀类型	后缀	示例（106）
长型（默认）	l	106l
无符号	u	106u
64位	ll（VS里是i64）	106ll

整数常数的表示

4-4 分数 5 查看题目详情

以下程序的输出结果是

5 分。

```
1 int main(void) {  
2     int a, b, c;  
3     a = 25;  
4     b = 025;  
5     c = 0x25;  
6     printf("%d %d %d\n", a, b, c);  
7     return 0;  
8 }
```

25 21 37

`a = 25`: 十进制, 值为 25。

`b = 025`: 八进制, 等于十进制的
($2 \times 8 + 5 = 21$)。

`c = 0x25`: 十六进制, 等于十进制的
($2 \times 16 + 5 = 37$)。

进制：看信息的角度

$$y_i = \frac{X}{M^i} \bmod M$$

9	0	2	7
9000	0	20	7

$$9 \cdot 10^3 + 0 \cdot 10^2 + 2 \cdot 10^1 + 7 \cdot 10^0$$

2	1	5	0	3
8192	512	320	0	3

$$2 \cdot 8^4 + 1 \cdot 8^3 + 5 \cdot 8^2 + 0 \cdot 8^1 + 3 \cdot 8^0$$

例如：将10进制的9027转换为8进制数

$$9 \cdot 10^3 + 0 \cdot 10^2 + 2 \cdot 10^1 + 7 \cdot 10^0 = 9027$$

$$9027 \div 8 = 1128 \text{ 余 } 3$$

$$1128 \div 8 = 141 \text{ 余 } 0$$

$$141 \div 8 = 17 \text{ 余 } 5$$

$$17 \div 8 = 2 \text{ 余 } 1$$

$$2 \div 8 = 0 \text{ 余 } 2$$

答案： $(9027)_{10} = (21503)_8$ 。

字符常数的表示

- 字符常数（大小写有关）

'<字符序列>'

- 字符常数的类型是整型（4字节），而不是字符型
- 字符序列是指：除单引号（'）、反斜杠（\）或者换行符以外的所有源字符集成员，或者转义序列

转义序列举例	格式	示例
简单转义序列	反斜杠（\），非x小写字母或标点	\a \b \f \n \r \t \v \' \" \\ \?
八进制转义序列	反斜杠（\），1~3位八进制数（01234567）	\101
十六进制转义序列	反斜杠（\），小写字母x，十六进制数（0123456789ABCDEF）	\x41


```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("%zd\n", sizeof('a'));;
```

```
    printf("%zd\n", sizeof("a"));;
```

```
    char b = 'a';
```

```
    printf("%zd\n", sizeof(b));;
```

```
    return 0;
```

```
}
```

1. sizeof('a') = 4: 字符常量 'a' 的类型为 int, 占用 4 字节。

2. sizeof("a") = 2: 字符串常量 "a" 包含字符 'a' 和结束符 \0 共 2 字节。

3. sizeof(b) = 1: 字符变量 b 的类型为 char, 占用 1 字节。

4

2

1

整数常数的表示形式

• 整数（含字符）常数的主要表示形式

赋值方法	示例	值	特点
十进制写法	var=65;	65	不以0开头
八进制写法	var=0101;	65	以0开头，但不紧接x或X，后续数字是0~7。
十六进制写法	var=0x20;	32	以0x或0X开头，后续数字是0~9，A~F。
可见字符	var='A';	65	以单引号为界，中间只有一个字符
转义序列字符	var='\b';	9	以单引号为界，中间以反斜杠开头，跟着一个字母
八进制字符	var='\101';	65	以单引号为界，中间以反斜杠开头，但不紧接x或X，后续数字是0~7。
十六进制字符	var='\x20';	32	以单引号为界，中间以\x或\x开头，后续数字是0~9，A~F。

整数常数的表示形式

1-16 分数 2 查看题目详情

已知字符 **B** 的ASCII码是66，那么也可以通过转义符 ****，用 **'\66'** 来表示字符常量 **B**。

☐ T

☐ F

F。

'\66' 表示字符 'f'，因为八进制 66 等于十进制 54。

2-14 分数 3 查看题目详情

若有说明语句：char s='\42'；则变量s_____。

☐ A. 包含一个字符

☐ B. 包含两个字符

☐ C. 包含三个字符

☐ D. 说明不合法，s的值不确定

A。

在 C 语言中，\42 是一个八进制转义字符，它表示 ASCII 码的八进制值 42。

题目中 char s 表示一个字符型变量，它只能存储一个字符。

转义序列（如 \42）在编译时会被解析为一个对应的字符，而非多个字符。

实数常数的表示

- 格式（大小写无关） `<正负号><小数常数>[指数][浮点后缀]`

小数常数序列	示例	值
数字序列+小数点+数字序列	30.106	30.106
小数点+数字序列	.106	0.106
数字序列+小数点	106.	106.0

指数部分序列	示例	值
e（大小写）+正负号（默认正）+数字序列	1.5E-2	0.015

浮点后缀	含义	示例（106）
l（默认）	double类型	-10.6l
f	float类型	10.6f

整型分类

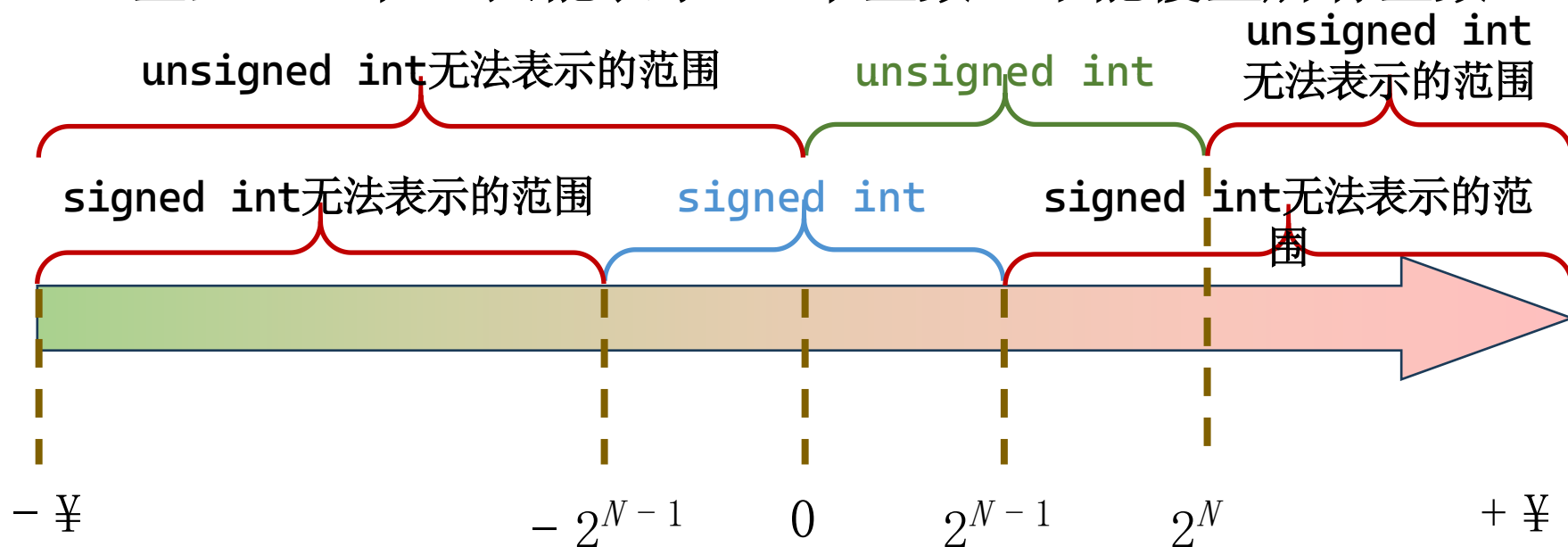
- 整型类型（按符号划分）
 - 有符号（signed，默认）、无符号（unsigned）
- 整型类型（按长度划分）
 - 字符型（char）：8位

字符型可以视为取值范围很小的整型
 - 短整型（short，16位机器int）：16位
 - 长整型（long，32和64位机器int）：32位
 - 超长整型（long long）：64位
 - 注意：在Linux GCC 64位编译器下，long被视为超长整型

整数的精度和范围

- 整型的精度是 1
- 整型不是整数

– 整型 (N 位) 只能表示 2^N 个整数, 不能覆盖所有整数



说明: 图示以 $N = 32$ 为例。

整数的精度和范围

- 整数内存占用: $N = 2^n$ bit
- 整数范围: 有符号 $[-2^{N-1}, 2^{N-1})$, 无符号 $[0, 2^N)$

名称	长度 ($N=2^n$)	有符号取值范围 ($[-2^{N-1}, 2^{N-1})$, 精度为1)	无符号取值范围 ($[0, 2^N)$, 精度为1)
字符型 ($n = 3$)	8 b	-128 ~ +127	0 ~ 255
短整型 ($n = 4$)	16 b	-32,768 ~ +32,767	0 ~ 65,535
长整型 ($n = 5$)	32 b	-2,147,483,648 ~ +2,147,483,647	0 ~ +4,294,967,295 (4×10^9)
超长整型 ($n = 6$)	64 b	-9,223,372,036,854,775,808 ~ +9,223,372,036,854,775,807	0 ~ +18,446,744,073,709,551,615 (1.8×10^{19})

整数的内存格式（字符型为例）

- 字符（character）

位置	7	6	5	4	3	2	1	0	值
二进制	1	0	1	1	0	0	1	0	-
十六进制	B				2				-
unsigned char	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	178
signed char	—	256-178=78							-78

位、字节和字

- 位（bit）：二元不确定性
 - 有2种情况概率相等，从未知到已知所获得的信息量为1位
 - 有 2^N 种情况概率相等，从未知到已知所获得的信息量为 N 位
- 字节（byte）：内存处理数据的最小单位
 - 1 byte = 8 bits
- 字（word）：用于一次性处理事务的固定长度位组
 - 现代计算机的字长通常为32、64位
 - 字长指寄存器的大小（即指示内存的范围）

位、字节和字

1-6 分数 2 查看题目详情

计算机采用的数制

在计算机内部，一切信息（包括数值、字符、指令等）的存储、处理与传送均采用十进制的形式。

☐ T

☐ F

F。

计算机内部的所有信息（包括数值、字符、指令等）的存储、处理与传送均采用二进制的形式，而不是十进制。

二进制使用 0 和 1 两个符号，适合通过电子方式（高低电压）进行表示和处理。

整数类型的溢出

- 整数类型的取值范围（如图）

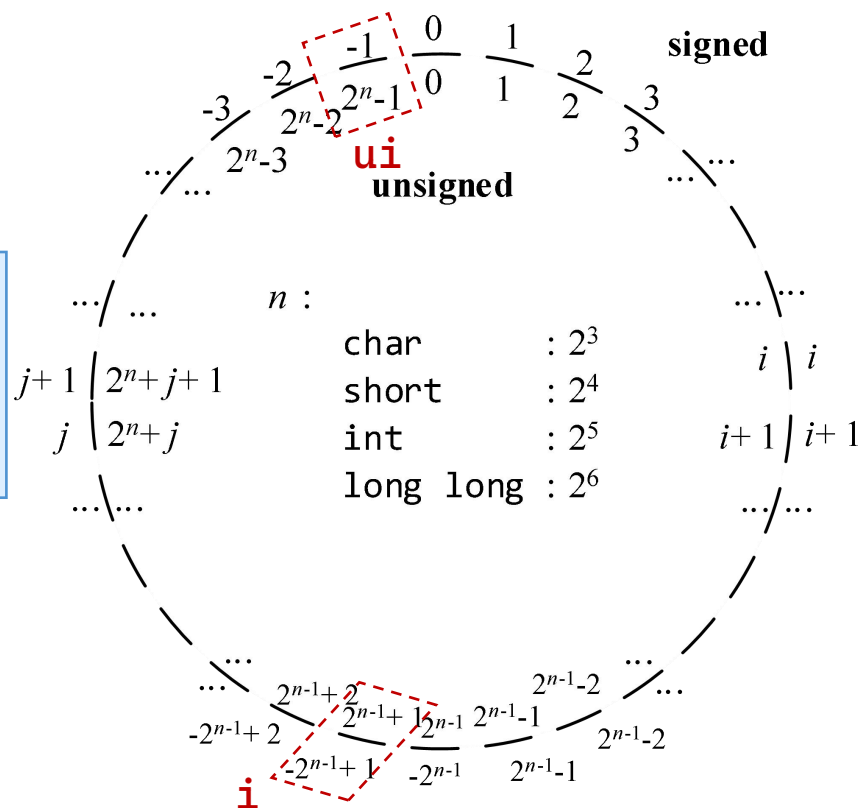
- 溢出：表达式超过了其数据类型的表示范围

- 溢出后的表现

- 根据图示自动转换

```
unsigned int ui = -1; 225 - 1  
int i = 2147483649u; 225-1 + 1  
printf("%u\n", i); // -2147483647
```

- 程序员应该避免溢出的发生



整数类型的溢出

C语言代码段如下:

```
1  int i = 32777;  
2  short si = i;  
3  int j = si;
```

short类型范围: **-32,768 ~ +32,767**

执行上述代码段后, **j** 的值是

- ☐ A. -32777 ☐ B. -32759 ☐ C. 32759 ☐ D. 32777

B。

1. **int i = 32777:** i 被初始化为 32777 (正常)。
2. **short si = i:** short 类型是 16 位, 有效范围是 -32768 到 32767。32777 超出范围, 发生溢出, 变成了 -32759。
3. **int j = si:** 将溢出后的值 -32759 赋给 j, 保持不变。

ASCII码表

扩展内容

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	_A	_B	_C	_D	_E	_F
0_	NUL 0000 0	SOH 0001 1	STX 0002 2	ETX 0003 3	EOT 0004 4	ENQ 0005 5	ACK 0006 6	BEL 0007 7	BS 0008 8	HT 0009 9	LF 000A 10	VT 000B 11	FF 000C 12	CR 000D 13	SO 000E 14	SI 000F 15
1_	DLE 0010 16	DC1 0011 17	DC2 0012 18	DC3 0013 19	DC4 0014 20	NAK 0015 21	SYN 0016 22	ETB 0017 23	CAN 0018 24	EM 0019 25	SUB 001A 26	ESC 001B 27	FS 001C 28	GS 001D 29	RS 001E 30	US 001F 31
2_	SP 0020 32	! 0021 33	" 0022 34	# 0023 35	\$ 0024 36	% 0025 37	& 0026 38	' 0027 39	(0028 40	) 0029 41	* 002A 42	+ 002B 43	, 002C 44	- 002D 45	. 002E 46	/ 002F 47
3_	0 0030 48	1 0031 49	2 0032 50	3 0033 51	4 0034 52	5 0035 53	6 0036 54	7 0037 55	8 0038 56	9 0039 57	: 003A 58	; 003B 59	< 003C 60	= 003D 61	> 003E 62	? 003F 63
4_	@ 0040 64	A 0041 65	B 0042 66	C 0043 67	D 0044 68	E 0045 69	F 0046 70	G 0047 71	H 0048 72	I 0049 73	J 004A 74	K 004B 75	L 004C 76	M 004D 77	N 004E 78	O 004F 79
5_	P 0050 80	Q 0051 81	R 0052 82	S 0053 83	T 0054 84	U 0055 85	V 0056 86	W 0057 87	X 0058 88	Y 0059 89	Z 005A 90	[005B 91	\ 005C 92	] 005D 93	^ 005E 94	_ 005F 95
6_	` 0060 96	a 0061 97	b 0062 98	c 0063 99	d 0064 100	e 0065 101	f 0066 102	g 0067 103	h 0068 104	i 0069 105	j 006A 106	k 006B 107	l 006C 108	m 006D 109	n 006E 110	o 006F 111
7_	p 0070 112	q 0071 113	r 0072 114	s 0073 115	t 0074 116	u 0075 117	v 0076 118	w 0077 119	x 0078 120	y 0079 121	z 007A 122	{ 007B 123	 007C 124	} 007D 125	~ 007E 126	DEL 007F 127

混淆数字字符和数字的概念

- 数字字符 '0' — '9' 和数值 0 — 9 不同
- 不同的格式说明符在键盘按下  时，解析的值不同

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	...
0_	NUL 0000 0	SOH 0001 1	STX 0002 2	ETX 0003 3	EOT 0004 4	ENQ 0005 5	ACK 0006 6	BEL 0007 7	BS 0008 8	HT 0009 9	...
...	字符 '0' — '9', 数值 48 — 57...										...
3_	0 0030 48	1 0031 49	2 0032 50	3 0033 51	4 0034 52	5 0035 53	6 0036 54	7 0037 55	8 0038 56	9 0039 57	...

说明符	数值	对应字符
%c	48	'0'
%d	0	'\0'

```
char c = 0;  
scanf("%c", &c);  
if (c == 0)  
    return 0;
```

```
char c = 0;  
scanf("%c", &c);  
if (c == '0')  
    return 0;
```

```
char c = 0;  
scanf("%d", &c);  
if (c == 0)  
    return 0;
```

混淆数字字符和数字的概念

2-11 分数 3 查看题目详情

已知字符 'a' 的ASCII码为 97，执行下列语句的输出是____。

```
printf ("%d, %c", 'b', 'b'+1 );
```

- ☐ A. 98, b
- ☐ B. 语法不合法
- ☐ C. 98, 99
- ☐ D. 98, c

D。

1. 'b' 的 ASCII 值是 98，因此 "%d" 对应的输出是 98。
2. 'b' + 1 相当于把 ASCII 值 98 加 1，变成 99，对应的字符是 'c'。
3. "%c" 对应字符输出，因此输出 'c'。

混淆数字字符和数字的概念

2-15 分数 3 查看题目详情

下列程序执行后的输出结果是

```
1 main()
2 {
3     int x='f';
4     printf("%c \n", 'A'+(x-'a'+1));
5 }
```

- ☐ A. G
- ☐ B. H
- ☐ C. I
- ☐ D. J

A。

1. `'x = 'f'`：这里 `'x'` 被赋值为字符 `'f'`。

2. 在表达式 `'x - 'a'` 中：

- 字符 `'f'` 的位置是字母表里的第6个字母。
- 字符 `'a'` 的位置是字母表里的第1个字母。
- 因此，`'x - 'a'` 表示的是 `'f'` 的字母位置减去 `'a'` 的字母位置，即结果是 `'6 - 1 = 5'`。

3. 接着计算 `'A' + (x - 'a' + 1)`：

- 字符 `'A'` 是字母表大写部分的第1个字母。
- `'x - 'a' + 1` 是 `'5 + 1 = 6'`。
- 将 `'A'` 为起点向后移动6位，变成大写字母的第6个字母，即 `'H'`。

浮点型分类

- 浮点型都是有符号的类型
- 浮点型类型（按长度划分）

$$\pm \text{尾数} * 2^{\text{指数}}$$

小数范围：
 $2^{23}=8388608$

关键字	名称	二进制位数				十进制位数	
		总	符号	指数	小数	小数精度	指数范围
float	单精度	32	1	8	23	-6 ~ -7	-37 ~ +38
double	双精度	64	1	11	52	-15 ~ -16	-307 ~ +308
long double	长双精度	80	1	15	64	-19 ~ -20	-4391 ~ +4392

* long double 为了对齐实际是12B或16B，不足补0。（初学者不需要掌握）

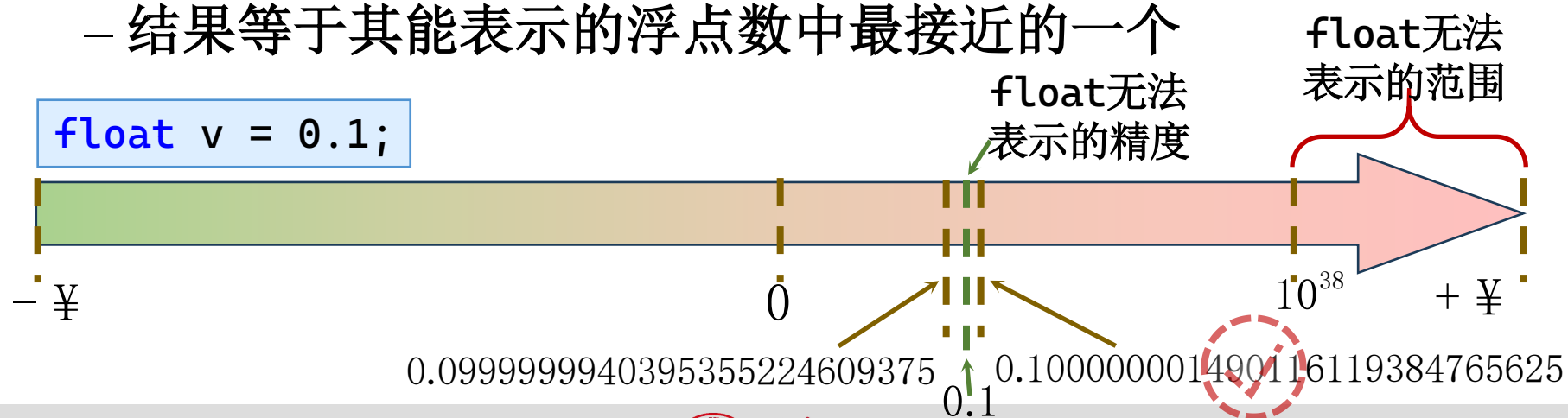
指数范围：

$$2^8=256$$

有符号型，所以-127~128（实际上去掉正负无穷-126~127）
- 2^{128} 到 2^{128} ，约等于- $3.4E38$ 到 $+3.4E38$

浮点型的精度

- 浮点型不是实数
 - 实数的范围是无穷大，精度也是无穷精度
 - 浮点型（ N 位）只能表示 2^N 个实数，不能覆盖所有实数
 - 浮点型表示范围由指数部分决定，精度由小数部分决定
- 浮点型变量常用有理数赋值，结果可能不如预期
 - 结果等于其能表示的浮点数中最接近的一个



忽视浮点型精度问题的错误

- 浮点型存在精度问题

- 运算结果应为 x 时，实际结果可能是 y

- 原因：十进制小数转换为二进制需要更多的有限数字表示

- 浮点型比较大小关系，会以一定概率出现答案错误

```
( 1.1f - 1.f - .1f ) == 0 // 1.1000000238
```

```
fabs( 1.1f - 1.f - .1f ) < 1e-6
```

$$1.099999904632568359375 = 1 \times 2^0 + 1 \times 2^{-4} + 1 \times 2^{-5} + 1 \times 2^{-8} + 1 \times 2^{-9} + 1 \times 2^{-12} \\ + 1 \times 2^{-13} + 1 \times 2^{-16} + 1 \times 2^{-17} + 1 \times 2^{-20} + 1 \times 2^{-21}$$

$$1.10000002384185791015625 = 1 \times 2^0 + 1 \times 2^{-4} + 1 \times 2^{-5} + 1 \times 2^{-8} + 1 \times 2^{-9} + 1 \times 2^{-12} \\ + 1 \times 2^{-13} + 1 \times 2^{-16} + 1 \times 2^{-17} + 1 \times 2^{-20} + 1 \times 2^{-21} + 1 \times 2^{-23}$$

$$0.0999999940395355224609375, 0.100000001490116119384765625$$

4

字符串和 格式化输入输出

理论课程



福州大学
FUZHOU UNIVERSITY

字符串常量

- 字符串特指由可见字符组成以0为结尾的数组
 - 字符串常量以一对双引号 " 为起止
 - 期间的转义字符 \" 除外
 - 以可见字符或转义字符的组成
 - 转义字符以反斜杠 \ 开始，紧随1个或多个字符，表示特殊字符
- 字符串的内存存储
 - 按每个字符存储，转义字符做转义处理，最后补结束标识0

"Hello \" world!"

字符	H	e	l	l	o	□	"	□	w	o	r	l	d	!	\\0
值	72	101	108	108	111	32	34	32	119	111	114	108	100	33	0

字符串基础知识

• 字符串的声明

格式	赋值语句
char 数组名[长度];	char c[60];
char 数组名[长度]="内容";	char c[60] = "Hello world!";
char 数组名[]="内容";	char c[] = "Hello world!";
char *数组名="内容";	char *c = "Hello world!";

- 长度应至少为字符串字符数加1。

此时c的元素不可改变

• 字符串的调用

存储在只读存储区（通常是内存中的只读段），只读存储区：通常是代码段或常量区，也称为只读区）。

功能	格式	表达式
表示字符串整体	数组名	c
表示字符串元素	数组名[下标]	c[3]

格式化输出函数 printf()

printf

功能	格式化输出	
格式	<code>int printf(const char *format, ...);</code>	
参数	<i>format</i>	格式字符串，指定后续参数的显示格式 格式字符串中包含格式说明符（以 % 开头）
	<i>...</i>	与格式标识符对应的变量列表（不定长）
返回值	成功	最终输出的字符个数
头文件	stdio.h	
说明	1. 对变量列表计算表达式的值，依次对应格式字符串中的格式说明符，生成字符串，替换原格式字符串部分。 2. 将替换后的字符串显示在标准输出流中。 3. 默认的标准输出流是显示器，但可以更改。	

格式字符串

- 格式字符串中包含若干个格式说明

Times字体的方括号表示该内容可选且可忽略

- 格式说明的形式:

%[标志][宽度][.精密度][修饰符]类型

– 示例:

%09.5lf

001.70000

成分	可选	说明
标志	可选	用于控制符号、空白、前导零点、小数点、八进制和十六进制前缀的输出对齐和输出。
宽度	可选	十进制整数，指定输出的最小字符数。
精密度	可选	它指定为字符串打印的最大字符数、浮点值小数点字符之后的有效位数或数字位数，或为整数值打印的最小位数。
修饰符	可选	它指定数据类型的长度。
类型	必选	确定相关联的参数被解释为字符、字符串、整数或浮点数。


```
// floats.c -- some floating-point combinations
#include <stdio.h>
int main(void)
{
    const double RENT = 3852.99;    // const-style constant
    printf("%f*\n", RENT);
    printf("%e*\n", RENT);
    printf("%4.2f*\n", RENT);
    printf("%3.1f*\n", RENT);
    printf("%10.3f*\n", RENT);
    printf("%10.3E*\n", RENT);
    printf("%+4.2f*\n", RENT);
    printf("%010.2f*\n", RENT);
    return 0;
}
```

不同类型和格式字符串输出

```
*3852.990000*
*3.852990e+003*
*3852.99*
*3853.0*
* 3852.990*
*3.853E+003*
*+3852.99*
*0003852.99*
```

```

/* flags.c -- illustrates some formatting flags */
#include <stdio.h>

int main(void)
{
    printf("%x %X %#x\n", 31, 31, 31);
    printf("**%d**% d**% d**\n", 42, 42, -42);
    printf("**%5d**%5.3d**%05d**%05.3d**\n", 6, 6, 6, 6);

    return 0;
}

```

十进制输出指定宽度最大最小值

```

1f 1F 0x1f
**42** 42**-42**
**      6**   006**00006**   006**

```

```
/* stringf.c -- string formatting */
```

```
#include <stdio.h>
```

```
#define BLURB "Authentic imitation!"
```

```
int main(void)
```

```
{
```

```
    printf("[%2s]\n", BLURB);
```

```
    printf("[%24s]\n", BLURB);
```

```
    printf("[%24.5s]\n", BLURB);
```

字符串输出指定宽度，左右对齐

```
    printf("[%24.5s]\n", BLURB);
```

```
    return 0;
```

```
[Authentic imitation!]
```

```
[    Authentic imitation!]
```

```
[                Authe]
```

```
[Authe                ]
```

```
/* prntval.c -- finding printf()'s return value */
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int bph2o = 212;
```

```
    int rv;
```

格式化输出的返回值是输出的字符个数

```
    rv = printf("%d F\n", bph2o);
```

```
    printf("The printf() function printed %d characters.\n", rv);
```

```
    return 0;
```

```
}
```

212 F

The printf() function printed 6 characters.

```
/* longstrg.c -- printing long strings */
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    printf("Here's one way to print a ");
```

```
    printf("long string.\n");
```

```
    printf("Here's another way to print a \n\n");
```

```
    printf("Here's the newest way to print a " "long string.\n");
```

```
/* ANSI C */
```

```
    return 0;
```

```
}
```

将一个字符串分行书写方便阅读代码，此时可以在分割处用反斜杠加回车表示，注意新行的字符串应顶格书写，否则会有多余的空格输出。

将一个字符串分行书写方便阅读代码，可以将其拆分成两个字符串相邻书写。

```
Here's one way to print a long string.
Here's another way to print a long string.
Here's the newest way to print a long string.
```

```
/* varwid.c -- uses variable-width output field */
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    unsigned width, precision;
```

```
    int number = 256;
```

```
    double weight = 242.5;
```

```
    printf("Enter a field width:\n");
```

```
    scanf("%d", &width);
```

```
    printf("The number is :%*d:\n", width, number);
```

```
    printf("Now enter a width and a precision:\n");
```

```
    scanf("%d %d", &width, &precision);
```

```
    printf("Weight = %*.*f\n", width, precision, weight);
```

```
    printf("Done!\n");
```

```
    return 0;
```

```
}
```

Enter a field width:

15↓

The number is : 256:

Now enter a width and a precision:

5 3↓

Weight = 242.500

Done!

用星号指定指定宽度和精度

格式化输入函数 scanf()

scanf

功能	格式化输入	
格式	<code>int scanf(const char *<i>format</i>, ...);</code>	
参数	<i>format</i>	格式字符串，指定后续参数的输入格式 格式字符串中包含格式说明符（以 % 开头）
	...	与格式标识符对应的变量列表（不定长）
返回值	成功	正确识别并赋值参数的个数
	失败	如果输入流关闭，返回-1（宏EOF）
头文件	stdio.h	
说明	1. 对变量列表计算表达式的值，从输入流中读取输入，依次对应格式字符串中的格式说明符，解析输入，如果成功解析则写入参数列表中的存储区域。	

格式化输入函数 scanf()

2-24 分数 1 查看题目详情

scanf()函数在读不到数据时返回 ()

- ☐ A. NULL
- ☐ B. 0
- ☐ C. EOF
- ☐ D. 1

C。

scanf() 函数的返回值:

如果成功读取了输入的参数, scanf() 返回成功读取的数据项数。

如果遇到输入失败(如文件结束或读取错误), scanf() 函数会返回 EOF。

EOF 的含义:

EOF 是一个特殊的常量, 表示 "End of File" (文件结束) 或出现错误。

格式输入函数的格式说明

• 格式说明的形式:

%[标志][宽度][修饰符]类型

– 示例:

%*5lf

格式说明符
标记

标志

宽度

大小前缀

类型

成分	可选	说明
标志	可选	仅星号（*），表示接受输入但忽略该项。
宽度	可选	十进制整数，指定输入的最大字符数，超过该长度，则不会被赋值给相应参数。
修饰符	可选	用于指示参数的大小。见格式输出函数的修饰符说明。
类型	必选	确定相关联的参数被解释为字符、字符串、整数或浮点数。

字符串的格式输入

- 通过 `scanf()` 配合 `%s` 输入一个单词

```
scanf("%s", name);
```

- 程序暂停直到用户输入回车为止

Wei HUANG ← 由于缓冲机制，输入回车之前程序暂停

- 格式输入在空白字符处停止，添加数字0，写入指定内存
 - 空格(blank)、制表符(tab)、换行符(newline)

偏移量	0	1	2	3	4 ... 39
字符	W	e	i	\0	该范围的值未改变
值	87	101	105	0	该范围的值未改变

- 如果输入要包括空格使用 `gets()` 函数

```
gets(name);
```

```

/* praise2.c */
// try the %u or %lu specifiers if your implementation
// does not recognize the %zd specifier
#include <stdio.h>
#include <string.h>      /* provides strlen() prototype */
#define PRAISE "You are an extraordinary being."
int main(void)
{
    char name[40];

    printf("What's your name? ");
    scanf("%s", name);
    printf("Hello, %s. %s\n", name, PRAISE);
    printf("Your name of %zd letters occupies %zd memory
cells.\n",
           strlen(name), sizeof name);

```

用宏代替一个字符串

%zd为输出空间大小专用，
非GCC编译器可能不支持

```
    printf("The phrase of praise has %zd letters ",
strlen(PRAISE));
    printf("and occupies %zd memory cells.\n", sizeof
PRAISE);

    return 0;
}
```

strlen是字符串长度函数
sizeof是求取常（变）量空间大小的操作符

What's your name? Wei Huang↓
Hello, Wei. You are an extraordinary being.
Your name of 3 letters occupies 40 memory cells.
The phrase of praise has 31 letters and occupies 32 memory cells.

计算字符串长度时不包括末尾的数字0，字符串常量空间包括末尾数字0

避免在格式字符串输入多余的字符

- 格式化输入的格式字符串应简单明了

```
scanf ( "%d,%d", &a , &b ) ; scanf ( "%s", s ) ;
```

- 正确输入为: □, 整数, □, 逗号, □, 整数, 回车符
- 输入会停留在解析错误的位置

□: 可选Tab或空格

输入	前后函数返回值		a值	b值	s内容
1, 3↵ abc↵	2	1	1	3	abc
, 3, 1↵	0	1	未赋值	未赋值	, 3, 1
1↵ 3↵	1	1	1	未赋值	3
1, 4a↵	2	1	1	4	a
1 5↵	1	1	1	未赋值	5

5

运算符、 表达式和语句

理论课程



福州大学
FUZHOU UNIVERSITY

算术运算符

- 算术运算的四则运算规则与算术课基本相同
- 参与四则运算的数据类型限定了范围和精度
- 运算的类型以两个操作数数据类型较高优先级为准
 - 低优先级的操作数先转换为该类型再计算，结果为该类型

优先级顺序	long double	double	float	long long int	int	short	char
-------	-------------	--------	-------	---------------	-----	-------	------

示例	两个操作数级别	值
<code>-7.3L+0.3f</code>	long double ✓, float	-6.999999988079
<code>-7.3f+4.0</code>	float, double ✓	-3.3
<code>-1u+3L</code>	unsigned int, long long int ✓	4294967298
<code>(short)65535*(char)3</code>	short ✓, char	-3 (溢出)

算术运算符

2-8 分数 1 查看题目详情

若有 `char w; int x; float y; double z;` 则表达式 `w*x+z-y` 值的数据类型为 ()。

- ☐ A. `float`
- ☐ B. `char`
- ☐ C. `int`
- ☐ D. `double`

D。

不同类型的提升规则：

`char` 和 `int` 会被提升为 `int`。

`int` 和 `float` 的运算结果为 `float`。

`float` 和 `double` 的运算结果为 `double`。

表达式 `w * x + z - y` 的类型推导过程：

`w * x`：

`w` 是 `char`，被提升为 `int`，`x` 也是 `int`，结果为 `int`。

`w * x + z`：

`int` 和 `double` 运算，结果为 `double`。

`w * x + z - y`：

`double` 和 `float` 运算，结果为 `double`。

最终结果为 `double` 类型。

算术运算符

- 操作数都是整数的除法，执行整除
 - 商和余数的值按被除数和除数绝对值相除
 - 商的符号按被除数与除数是否同号，余数符号同被除数

int X	int Y	X/Y	X%Y
7	4	1	3
-7	4	-1	-3
7	-4	-1	3
-7	-4	1	-3

X	Y	X/Y
7	4	1
7.0	4	1.75
7	4.0	1.75
7.0	4.0	1.75

- 取模（余数）的操作数仅限整数，否则出现编译错误

[Error] invalid operands to binary % (have 'double' and 'int')

```

#include <stdio.h>
int main()
{
    double result ;
    result = 5 / 2;
    printf("%lf\n", result);
    result = 5 / 2.;
    printf("%lf\n", result);
    result = 1. * 5 / 2;
    printf("%lf\n", result);
    result = (double)(5) / 2;
    printf("%lf\n", result);
    return 0;
}

```

```

2.000000
2.500000
2.500000
2.500000

```

算术运算符

2-4 分数 1 查看题目详情

表达式_____的值是0。

- ☐ A. $3\%5$
- ☐ B. $3/5.0$
- ☐ C. $3/5$
- ☐ D. $3<5$

C。

$3/5$ 是整数除法，结果为 0
(因为 3 和 5 都是整数，整数除法舍去小数部分)。

2-6 分数 1 查看题目详情

若 x 为浮点型，则表达式 $x=10/4$ 为 () 。

- ☐ A. 2.5
- ☐ B. 2.0
- ☐ C. 2
- ☐ D. 3

B。

表达式中虽然 x 是浮点型，但表达式 $10/4$ 中，10 和 4 都是整数，所以这是整数除法。整数除法会直接去掉小数部分， $10/4=2$ 。然后再赋值给浮点型变量 x ，变成了 2.0。

如果要得到 2.5，必须让其中一个数是浮点数，比如 $10.0/4$ 或 $10/4.0$ 。

赋值运算符

赋值

功能	计算右值，做为左值的新值	
格式	$lvalue = rvalue$	
参数	$lvalue$	左值，限于：变量名、数组元素、结构体成员
	$rvalue$	右值，可以是表达式。
求值	右值的值	
副作用	改变了左值的值	
示例	$x = 32 + 5$	整体值为37，运行后x值为37
说明	$A = B$ 不表示“A等于B”	

```
/* golf.c -- golf tournament scorecard */
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int jane, tarzan, cheeta;
```

等价于:

jane=68;

tarzan=jane;

cheeta=tarzan;

```
    cheeta = tarzan = jane = 68;
```

```
    printf("                cheeta    tarzan    jane\n");
```

```
    printf("First round score %4d %8d %8d\n", cheeta, tarzan, jane);
```

```
    return 0;
```

```
}
```

	cheeta	tarzan	jane
First round score	68	68	68

复合赋值

复合赋值

格式	$lvalue \ += \ rvalue$	计算 $lvalue = lvalue + rvalue$ 。
	$lvalue \ -= \ rvalue$	计算 $lvalue = lvalue - rvalue$ 。
	$lvalue \ *= \ rvalue$	计算 $lvalue = lvalue * rvalue$ 。
	$lvalue \ /= \ rvalue$	计算 $lvalue = lvalue / rvalue$ 。
	$lvalue \ \% \ rvalue$	计算 $lvalue = lvalue \% rvalue$ 。
求值	计算 $lvalue \ op \ rvalue$ 的值	
副作用	用 $lvalue \ op \ rvalue$ 的值改变 $lvalue$ 的值。	
示例	$c \ /= \ 5.0$	设运行前 c 为 6.4，整体值为 1.28 运行后， c 的值为 1.28
说明	1. 运算规则（尤其除、模等），和其等价的展开式相同。 2. $lvalue$ 的左值规则，与普通赋值符号的规则相同。	

复合赋值

- 复合赋值与其展开式

- 书写程序时使用复合赋值替代其展开式，显得更加简洁
- 预估复合赋值的运算结果时，用其展开式代替

- 左值和右值的约束条件，与其展开式的约束相同。

- 左值限于：变量名、数组元素、结构体成员
- 除法赋值：左值右值都是整型时，整除；否则，实数除
- 取模赋值：限于左值和右值是整型

表达式 (int n; double d;)	运行	表达式	运行
n /= 3;	执行整除	(d + 5) /= 3;	编译错误
d /= 3;	执行实数除法	d %= 3;	编译错误
n %= 3;	执行取模	3 /= 2;	编译错误

强制类型转换

强制类型转换

功能	将操作数强制转换为新类型	
格式	<code>(type)exp1</code>	
参数	<code>type</code>	类型名，需要强制转换的目标类型
	<code>exp1</code>	待转换的操作数
求值	强制类型转换后的值	
示例	<code>(int)32.5</code>	整体值为32
说明	强制类型转换可能导致精度和范围截断	

强制类型转换

- 强制类型转换与自动类型转换的区别联系
 - 自动类型转换通常是从数据范围较小的类型自动转换为数据范围较大的类型，不会丢失精度和范围
 - 强制类型转换可能会丢失精度或范围
- 如不强制类型转换结果也正确，不必强制类型转换

表达式语句（mice为double类型）	运行后mice的值	说明
<code>mice = 1.6 + 1.7 + 1.8;</code>	5.1	直接相加。
<code>mice = (int)1.6 + 1.7 + (int)1.8;</code>	3.7	第1、3项值为1。
<code>mice = (int)(1.6 + 1.7) + (int)1.8;</code>	4.0	第1项值为3。
<code>mice = int(1.6 + 1.7) + int(1.8);</code>	编译错误	这是C++的语法，不是C的语法。

强制类型转换

1-19 分数 1 查看题目详情

若有定义：int a=2,b=3; float x=3.5,y=2.5;则表达式(float)(a+b)/2+(int)x%(int)y的值是3。

☐ T

☐ F

F。

$$(\text{float})(a + b) / 2 + (\text{int})x \% (\text{int})y = 2.5 + 1 = 3.5$$

尺寸运算符

尺寸

功能	用于返回表达式或数据类型的大小	
格式	<code>sizeof(type)</code> 或 <code>sizeof(exp)</code> 或 <code>sizeof exp</code>	
参数	<code>type</code>	类型名，需要求尺寸的类型，该项括号是必须的
	<code>exp</code>	表达式，需要求尺寸的表达式
求值	计算类型的尺寸，或表达式对应类型的尺寸，计算结果对应数据类型为 <code>size_t</code>	
示例	<code>sizeof(double)</code>	整体值为8
说明	1. 表达式不会被计算，表达式的副作用不会生效。 2. 不同程序架构和编译器下，一些类型的值结果可能不同。	

尺寸运算符

- 注意事项

- 尺寸运算符后的表达式，**不计算，副作用也不生效**

```
int a = 4;  
int s = sizeof(a=3);
```

此处a=3不会生效，仅计算a=3对应类型的尺寸。运行后a为4，s为4。

- 同一类型的尺寸与应用程序架构有关

- 在GCC 64位程序中，sizeof(long)是8，一般情况是4。

- size_t类型尺寸与应用程序架构有关

- 在32位应用程序中，size_t是32位无符号整数

- 在64位应用程序中，size_t是64位无符号整数

```
// sizeof.c -- uses sizeof operator
// uses C99 %z modifier -- try %u or %lu if you lack %zd
#include <stdio.h>
int main(void)
{
    int n = 0;
    size_t intsize;

    intsize = sizeof (int);
    printf("n = %d, n has %zd bytes; all ints have %zd bytes.\n",
        n, sizeof n, intsize );

    return 0;
}
```

有经验的程序员不会对同一类量使用不同的实现风格。

```
n = 0, n has 4 bytes; all ints have 4 bytes.
```

尺寸运算符

1-15 分数 1 查看题目详情

执行以下程序段后，**k**的值为4。

```
int k = 3;  
sizeof(++k);
```

☐ T

☐ F

F。

sizeof 是一个运算符，不会计算表达式的值：

在 sizeof(++k) 中，++k 不会执行递增操作，因为 sizeof 在编译时只处理数据类型，而不实际运行表达式。

所以，k 的值不会被改变，依旧为 3。

增量减量运算符

- 前缀和后缀增减量的区别

- 前缀**先增减量**再求值；后缀先求值**再增减量**

表达式的值和 $exp1$ 的值不是一回事

名称	表达式	表达式的值	运行后 $exp1$ 的值
前缀增量运算符	$++exp1$	$exp1+1$ 的值	$exp1+1$ 的值
后缀增量运算符	$exp1++$	$exp1$ 的值	$exp1+1$ 的值
前缀减量运算符	$--exp1$	$exp1-1$ 的值	$exp1-1$ 的值
后缀减量运算符	$exp1--$	$exp1$ 的值	$exp1-1$ 的值

- 程序员编程时对增量减量的使用应**适可而止**

- 程序员有义务让读者看懂（**除非你要保护技术秘密**）
 - 适当合并表达式简化代码，但过度合并难以阅读

```
ans /= num / 2 + 6 * ( 1 + num++ + ++num ); // ans=7, num=3
```

```
/* post_pre.c -- postfix vs prefix */
```

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int a = 1, b = 1;
```

```
    int a_post, pre_b;
```

a的值是1，所以a++的值是1，
a_post的值是1，运行后a的值是2

b的值是1，所以++b的值是2，
pre_b的值是2，运行后b是2

```
    a_post = a++; // value of a++ during assignment phase
```

```
    pre_b = ++b; // value of ++b during assignment phase
```

```
    printf("a  a_post  b  pre_b \n");
```

```
    printf("%1d %5d %5d %5d\n", a, a_post, b, pre_b);
```

```
    return 0;
```

```
}
```

a	a_post	b	pre_b
2	1	2	2

关系运算符

关系运算

格式	$exp1 < exp2$	判断 $exp1$ 是否小于 $exp2$
	$exp1 \leq exp2$	判断 $exp1$ 是否小于或等于 $exp2$
	$exp1 > exp2$	判断 $exp1$ 是否大于 $exp2$
	$exp1 \geq exp2$	判断 $exp1$ 是否大于或等于 $exp2$
	$exp1 == exp2$	判断 $exp1$ 是否等于 $exp2$
	$exp1 != exp2$	判断 $exp1$ 是否不等于 $exp2$
求值	判断 $exp1$ 和 $exp2$ 的具体关系。为真，值为1；为假，值为0。	
示例	$3 \geq 3$	值为1
说明	1. 大（或小）于（等）于号的运算优先级高于（不）等号。 2. 浮点数存在舍入误差，慎用相等运算符。	

```
/* t_and_f.c -- true and false values in C */
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int true_val, false_val;
```

通过这个程序可以查看真和假在参与运算时的值。

```
    true_val = (10 > 2);    // value of a true relationship
```

```
    false_val = (10 == 2); // value of a false relationship
```

```
    printf("true = %d; false = %d \n", true_val, false_val);
```

```
    return 0;
```

```
}
```

```
true = 1; false = 0
```

逻辑运算符

逻辑与或

格式	<code>exp1 && exp2</code>	判断 <code>exp1</code> 和 <code>exp2</code> 是否同时为真
	<code>exp1 exp2</code>	判断 <code>exp1</code> 和 <code>exp2</code> 是否不同时为假
求值	判断 <code>exp1</code> 和 <code>exp2</code> 的逻辑关系。 非零与非零为1，其余为0；零或零为0，其余为1。	
示例	<code>3 && 3</code>	值为1
	<code>5 && 0</code>	值为0
	<code>0 6</code>	值为1
	<code>0 0</code>	值为0
说明	- 参与逻辑运算的两个表达式，非0表示真，0表示假。 - 计算结果，真为1，假为0。 - 短路计算：<code>exp1</code>为假时，逻辑与的<code>exp2</code>不会被计算；<code>exp1</code>为真时，逻辑或的<code>exp2</code>不会被计算。	

逻辑运算符

- 逻辑与相当于算术乘法，逻辑或相当于算术加法

1	&&	1	→	1
0	&&	1	→	0
1	&&	0	→	0
0	&&	0	→	0

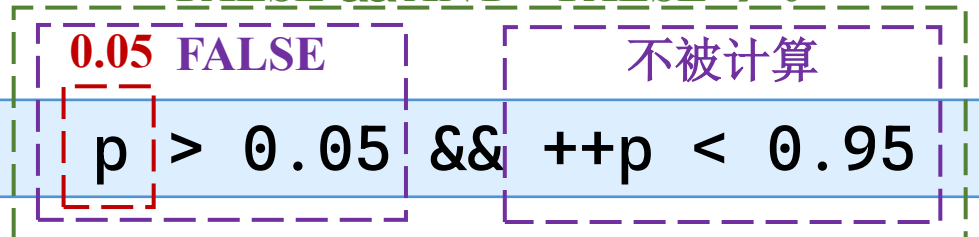
1		1	→	1
0		1	→	1
1		0	→	1
0		0	→	0

逻辑表达式的短路计算

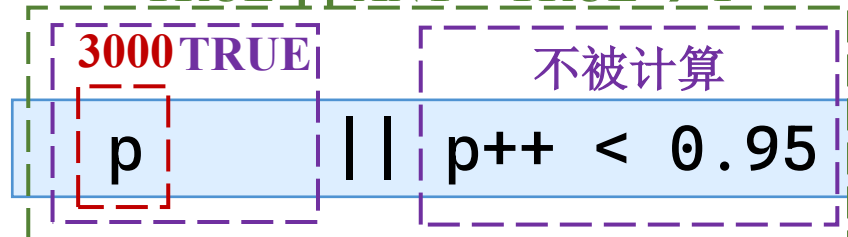
- 左侧表达式为假时，逻辑与的右侧表达式不会被计算
 - 假与任何数都为假，该情况右侧表达式不需要计算
- 左侧表达式为真时，逻辑或的右侧表达式不会被计算
 - 真或任何数都为真，该情况右侧表达式不需要计算

名称	运算符	伪代码描述	短路计算规则
逻辑与	$x \ \&\& \ y$	if (x) y else FALSE	x等于0时，y不计算。
逻辑或	$x \ \ y$	if (x) TRUE else y	x不等于0时，y不计算。

FALSE && ANY = FALSE → 0



TRUE || ANY = TRUE → 1



逻辑运算符

逻辑否

格式	<code>!exp1</code>	判断 <code>exp1</code> 是否为假
求值	判断 <code>exp1</code> 是否为0。 <code>exp1</code> 为0，则值为1； <code>exp1</code> 非0，则值为0。	
示例	<code>!3</code>	值为0
	<code>!0</code>	值为1
说明	- 参与逻辑运算的表达式，非0表示真，0表示假。 - 计算结果，真为1，假为0。	

逻辑运算符

1-6 分数 1 查看题目详情

只有数字1代表逻辑值“真”。

☐ T

☐ F

1-14 分数 1 查看题目详情

语句:printf("%d",(a=2)&&(b=-2));的输出结果是1。

☐ T

☐ F

F。

在C语言中，任意非零数字都代表逻辑值“真”，而不仅仅是数字1。数字0代表逻辑值“假”。

T。

a=2 和 b=-2 是赋值操作，返回的值分别是2和-2。

逻辑与运算符 &&：

a=2 的结果是非零，逻辑值为“真”。

b=-2 的结果也是非零，逻辑值也为“真”。

两个都为真，因此 (a=2) && (b=-2) 的结果是1。

printf 输出：

%d 格式化输出的是整数值，最终输出结果为1。

逻辑运算符

2-30 分数 1 查看题目详情

设有定义“int a=1, b=2, c=3, d=4, m=2, n=2;”, 则执行表达式“(m=a>b)&&(n=c>d)”后,n的值为()。

- ☐ A. 1
- ☐ B. 2
- ☐ C. 3
- ☐ D. 0

B。

第一部分 ($m = a > b$):

比较 $a > b$:

$a = 1$, $b = 2$ 。因为 $1 > 2$ 为假, 结果为 0。

给 m 赋值:

$m = 0$, 此时 m 的值变为 0。

短路运算:

&& 是逻辑与运算符:

短路特性: 如果左表达式的值为假 (即 $m = 0$), 右表达式 ($n = c > d$) 不会再执行。

因此, 右表达式 不会修改 n 的值, n 的值保持原来的值。

逗号运算符

逗号

格式	$exp1, exp2$	计算 $exp1$ 和 $exp2$ 的值
求值	表达式的值为 $exp2$ 的值。	
示例	2, 1	值为1
	x = 1, 3	值为3
说明	无。	

逗号运算符

1-20

分数 1

[查看题目详情](#)

表达式 $(j=3, j++)$ 的值是4.

☐ T

☐ F

F。

表达式 $(j=3, j++)$ 是一个逗号表达式。逗号表达式中，最终的结果是最后一个子表达式的值，其执行过程如下：

$j = 3$:

把值 3 赋给变量 j ，此时 $j = 3$ 。

$j++$:

后置自增操作，返回值是 自增前的值，即返回 3，然后将 j 的值增加 1。

此时 j 的值变为 4。

逗号表达式最终的值：

逗号表达式的结果是最后一个子表达式的值，即 $j++$ 原来的值是 3。

条件运算符

条件运算符

格式	$exp1?exp2:exp3$	根据 $exp1$ 的值计算 $exp2$ 或 $exp3$ 的值
求值	如果 $exp1$ 的值非0，则表达式取 $exp2$ 的值，且 $exp3$ 不被计算。 如果 $exp1$ 的值为0，则表达式取 $exp3$ 的值，且 $exp2$ 不被计算。	
示例	$(x == 2)?4:0$	如果运行前 x 值为1，则表达式值为0
	$(x == 2)?4:0$	如果运行前 x 值为2，则表达式值为4
说明	1. 表达式 $(x)?y:z$ 相当于 <code>if (x) y else z</code>。 2. 无关的表达式不会被计算。	

运算符的优先级

- 下表为与本节相关的优先级顺序

– 一元>二元>三元>赋值>逗号, 算术>关系>逻辑

序号	符号	说明	序号	符号	说明
1 →	后缀++ --	后缀增减量	4	+ -	算术运算：加减
	()	函数调用	6	< > <= >=	关系运算符：大小
	[]	数组下标	7	== !=	关系运算符：相等
2 ←	前缀++ --	前缀增减量	11	&&	逻辑运算符：与
	+ -	正负号	12		逻辑运算符：或
	!	逻辑运算符：非	13	?:	三元条件运算符
	(type)	强制类型转换	14 ←	=	赋值
	sizeof	存储空间		+= -= *= /= %=	复合运算符
3	* / %	算术运算：乘除模	15	,	逗号表达式

语句

• C语言语句的分类

语句

声明语句

```
int x, y, z;
```

标签语句

```
stop: printf("Stop!\n");
```

复合语句

```
{ x=5; y=6; }
```

表达式语句

```
y=0.3*x+6+eps;
```

循环语句

```
while (x<3) x++;
```

选择语句

```
if (x>5) y=1; else y=4;
```

跳转语句

```
if (x==0) break;
```

空语句

```
;
```

返回语句

```
return 0;
```

语句

1-10 分数 2 查看题目详情

`#define PI 3.1415926` 是一条C语句。

☐ T

☐ F

F。

`#define PI 3.1415926` 是一条预处理指令，不是严格意义上的 C 语言语句。它属于 宏定义指令，用于在编译前用 3.1415926 替换代码中出现的 `PI`。C 语言语句 是指在程序运行时由编译器生成机器代码并执行的语句（如赋值、循环等）。而预处理指令在编译阶段就被处理完，并不会生成机器代码。

F 宏属于C语言的预处理功能，在编译之前要替换到程序中。

谢谢观看

